

Python Practical Notes

Lambda, Map/Filter/Reduce, Comprehensions, Modules, Exception Handling, File Handling & OOP

Recommended Learning Order

1. Lambda Functions
2. map()
3. filter()
4. reduce()
5. List Comprehension
6. Dictionary Comprehension
7. Modules & Imports
8. Packages
9. Exception Handling
10. File Handling
11. CSV Files
12. OOP Fundamentals
13. Encapsulation
14. Inheritance
15. Polymorphism
16. Abstraction
17. Method Overriding
18. Mini Project
19. Assignments with Solutions

1. Lambda Functions

1.1 What is Lambda?

A **lambda function** is a small anonymous function.

Normal function:

```
def square(x):
```

```
    return x * x
```

Lambda equivalent:

```
square = lambda x: x * x
```

```
print(square(5))
```

Output:

25

Syntax

lambda arguments: expression

Example:

```
lambda x: x * 2
```

Here:

- lambda → keyword
- x → parameter
- x * 2 → expression/result

1.2 Lambda with Multiple Arguments

```
add = lambda a, b: a + b
```

```
print(add(10, 20))
```

Output:

30

Another example:

```
multiply = lambda a, b: a * b
```

```
print(multiply(5, 4))
```

Output:

1.3 Lambda with Condition

```
check = lambda x: "Even" if x % 2 == 0 else "Odd"
```

```
print(check(10))
```

```
print(check(7))
```

Output:

Even

Odd

2. map()

2.1 What is map()?

map() applies a function to **every element** of an iterable.

Syntax

```
map(function, iterable)
```

Example:

```
numbers = [1, 2, 3, 4, 5]
```

```
result = map(lambda x: x * 2, numbers)
```

```
print(list(result))
```

Output:

```
[2, 4, 6, 8, 10]
```

How it works

$1 \rightarrow 1 \times 2 \rightarrow 2$

$2 \rightarrow 2 \times 2 \rightarrow 4$

$3 \rightarrow 3 \times 2 \rightarrow 6$

$4 \rightarrow 4 \times 2 \rightarrow 8$

$5 \rightarrow 5 \times 2 \rightarrow 10$

2.2 map() with Normal Function

```
def square(x):
```

```
    return x * x
```

```
numbers = [1, 2, 3, 4, 5]
```

```
result = map(square, numbers)
```

```
print(list(result))
```

Output:

```
[1, 4, 9, 16, 25]
```

2.3 Practical Example

Suppose student marks are:

```
marks = [45, 67, 78, 89, 91]
```

Add 5 grace marks:

```
marks = [45, 67, 78, 89, 91]
```

```
new_marks = list(map(lambda x: x + 5, marks))
```

```
print(new_marks)
```

Output:

```
[50, 72, 83, 94, 96]
```

3. filter()

3.1 What is filter()?

filter() is used to **select elements based on a condition**.

Syntax

```
filter(function, iterable)
```

The function should return:

True → keep element

False → remove element

3.2 Example

Find even numbers:

```
numbers = [1, 2, 3, 4, 5, 6]
```

```
result = filter(lambda x: x % 2 == 0, numbers)
```

```
print(list(result))
```

Output:

```
[2, 4, 6]
```

3.3 Find Students Who Passed

```
marks = [35, 20, 67, 89, 28, 45]
```

```
passed = list(filter(lambda x: x >= 35, marks))
```

```
print(passed)
```

Output:

```
[35, 67, 89, 45]
```

3.4 Filter Names

```
names = ["Amit", "Raj", "Ankit", "Riya", "Rahul"]
```

```
result = list(filter(lambda name: len(name) > 4, names))
```

```
print(result)
```

Output:

```
['Amit', 'Ankit', 'Riya', 'Rahul']
```

4. reduce()

4.1 What is reduce()?

reduce() repeatedly applies a function to elements and produces **one final result**.

It is available in:

functools

So we need:

```
from functools import reduce
```

4.2 Example: Sum

```
from functools import reduce
```

```
numbers = [1, 2, 3, 4, 5]
```

```
result = reduce(lambda a, b: a + b, numbers)
```

```
print(result)
```

Output:

15

Internal working

1 + 2 = 3

3 + 3 = 6

6 + 4 = 10

10 + 5 = 15

4.3 Find Product

```
from functools import reduce
```

```
numbers = [1, 2, 3, 4, 5]
```

```
result = reduce(lambda a, b: a * b, numbers)
```

```
print(result)
```

Output:

120

5. Difference Between map(), filter() and reduce()

Function	Purpose	Result
map()	Transform every element	Multiple values
filter()	Select elements	Multiple values
reduce()	Combine elements	One value

Example:

```
numbers = [1, 2, 3, 4, 5]
```

map

```
[2, 4, 6, 8, 10]
```

filter

```
[2, 4]
```

reduce

15

6. List Comprehension

6.1 What is List Comprehension?

List comprehension provides a short way to create lists.

Normal approach:

```
numbers = []
```

```
for i in range(1, 6):
```

```
    numbers.append(i)
```

```
print(numbers)
```

Using comprehension:

```
numbers = [i for i in range(1, 6)]
```

```
print(numbers)
```

Output:

```
[1, 2, 3, 4, 5]
```

6.2 Syntax

```
[expression for item in iterable]
```

Example:

```
squares = [x * x for x in range(1, 6)]
```

```
print(squares)
```

Output:

```
[1, 4, 9, 16, 25]
```

7. List Comprehension with Condition

Find even numbers

```
numbers = range(1, 11)
```

```
even_numbers = [x for x in numbers if x % 2 == 0]
```

```
print(even_numbers)
```

Output:

```
[2, 4, 6, 8, 10]
```

Find numbers greater than 50

```
marks = [30, 55, 78, 42, 91, 65]
```

```
result = [mark for mark in marks if mark > 50]
```

```
print(result)
```

Output:

```
[55, 78, 91, 65]
```

8. Dictionary Comprehension

Dictionary comprehension is used to create dictionaries quickly.

Normal approach

```
squares = {}
```

```
for x in range(1, 6):
```

```
    squares[x] = x * x
```

```
print(squares)
```

Output:

```
{1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

Using comprehension:

```
squares = {x: x * x for x in range(1, 6)}
```

```
print(squares)
```

8.1 Dictionary with Condition

```
marks = {
```

```
    "Amit": 85,
```

```
    "Rahul": 32,
```

```
    "Riya": 91,
```

```
    "Ankit": 25
```

```
}
```

```
passed = {
```

```
    name: mark
```

```
    for name, mark in marks.items()
```

```
    if mark >= 35
```

```
}
```

```
print(passed)
```

Output:

```
{'Amit': 85, 'Riya': 91}
```

9. Modules

9.1 What is a Module?

A module is a Python file containing reusable code.

For example:

math_utils.py

contains:

```
def add(a, b):
```

```
    return a + b
```

Another file can use it.

10. Creating Your Own Module

Create this folder:

python_modules/

|

└─ main.py

└─ calculator.py

calculator.py

```
def add(a, b):
```

```
    return a + b
```

```
def subtract(a, b):
```

```
    return a - b
```

```
def multiply(a, b):
```

```
    return a * b
```

main.py

```
import calculator
```

```
print(calculator.add(10, 20))
```

```
print(calculator.subtract(20, 5))
```

```
print(calculator.multiply(5, 4))
```

Output:

```
30
```

```
15
```

```
20
```

11. Different Import Styles

Import complete module

```
import math
```

```
print(math.sqrt(25))
```

Import specific function

```
from math import sqrt
```

```
print(sqrt(25))
```

Import multiple functions

```
from math import sqrt, factorial
```

```
print(sqrt(25))
```

```
print(factorial(5))
```

Alias

```
import math as m
```

```
print(m.sqrt(25))
```

12. Built-in Modules

Python provides many useful modules.

Some examples:

math

random

datetime

os

sys

statistics

math

import math

print(math.sqrt(16))

print(math.pi)

random

import random

number = random.randint(1, 100)

print(number)

datetime

from datetime import datetime

now = datetime.now()

print(now)

13. Packages

What is a Package?

A package is a **folder containing related Python modules**.

Example:

my_project/

|

├── main.py

|

└── utilities/

 ├── __init__.py

 ├── calculator.py

 └── message.py

calculator.py

```
def add(a, b):
```

```
    return a + b
```

message.py

```
def welcome(name):
```

```
    return f"Welcome {name}"
```

main.py

```
from utilities.calculator import add
```

```
from utilities.message import welcome
```

```
print(add(10, 20))
```

```
print(welcome("Amit"))
```

14. Exception Handling

14.1 What is an Exception?

An exception is an error that occurs **while the program is running**.

Example:

```
a = 10
```

```
b = 0
```

```
print(a / b)
```

This produces:

ZeroDivisionError

Without exception handling, the program may stop.

15. try and except

Syntax:

try:

```
    # risky code
```

except:

```
    # error handling
```

Example:

try:

```
    a = 10
```

```
    b = 0
```

```
    print(a / b)
```

except:

```
    print("Cannot divide by zero")
```

Output:

Cannot divide by zero

16. Handling Specific Exceptions

Instead of using a general except, specify the error.

try:

```
    number = int(input("Enter number: "))
```

```
result = 10 / number
```

```
print(result)
```

```
except ValueError:
```

```
    print("Please enter a valid number")
```

```
except ZeroDivisionError:
```

```
    print("Number cannot be zero")
```

This is better because different errors can be handled differently.

17. else

else executes when **no exception occurs**.

```
try:
```

```
    number = int(input("Enter number: "))
```

```
except ValueError:
```

```
    print("Invalid input")
```

```
else:
```

```
    print("You entered:", number)
```

18. finally

finally executes **whether an exception occurs or not**.

```
try:
```

```
    print("Program running")
```

except:

```
print("Error occurred")
```

finally:

```
print("Program finished")
```

19. Complete Exception Handling Structure

try:

```
# risky code
```

except SomeError:

```
# handle error
```

else:

```
# executes when no error
```

finally:

```
# always executes
```

20. raise

raise is used when we want to **manually generate an exception**.

Example:

```
age = int(input("Enter age: "))
```

if age < 18:

```
raise ValueError("Age must be 18 or above")
```

```
print("Eligible")
```

21. Custom Validation

```
def withdraw(balance, amount):
```

```
    if amount <= 0:
```

```
        raise ValueError("Amount must be greater than zero")
```

```
    if amount > balance:
```

```
        raise ValueError("Insufficient balance")
```

```
    return balance - amount
```

```
try:
```

```
    balance = 5000
```

```
    amount = int(input("Enter withdrawal amount: "))
```

```
    balance = withdraw(balance, amount)
```

```
    print("Remaining balance:", balance)
```

```
except ValueError as e:
```

```
    print("Error:", e)
```

22. File Handling

File handling allows Python programs to:

- Create files

- Read files
 - Write files
 - Append data
 - Process CSV files
-

23. Opening a File

Basic syntax:

```
open("filename", "mode")
```

Common modes:

Mode Meaning

r Read

w Write

a Append

x Create

rb Read binary

wb Write binary

24. Writing to a Text File

```
file = open("students.txt", "w")
```

```
file.write("Amit\n")
```

```
file.write("Rahul\n")
```

```
file.write("Riya\n")
```

```
file.close()
```

This creates:

students.txt

25. Reading a File

```
file = open("students.txt", "r")
```

```
content = file.read()
```

```
print(content)
```

```
file.close()
```

26. Reading Line by Line

```
file = open("students.txt", "r")
```

```
for line in file:
```

```
    print(line.strip())
```

```
file.close()
```

27. readline()

```
file = open("students.txt", "r")
```

```
print(file.readline())
```

```
print(file.readline())
```

```
file.close()
```

28. readlines()

```
file = open("students.txt", "r")
```

```
lines = file.readlines()
```

```
print(lines)
```

```
file.close()
```

29. Append Mode

a adds data at the end.

```
file = open("students.txt", "a")
```

```
file.write("Ankit\n")
```

```
file.close()
```

Existing content is not deleted.

30. Using with

The recommended approach is:

with open("students.txt", "r") as file:

```
    content = file.read()
```

```
print(content)
```

The file is automatically closed.

Why use with?

Instead of:

```
file.close()
```

Python handles closing automatically.

31. File Exceptions

What happens if the file doesn't exist?

try:

```
with open("unknown.txt", "r") as file:
```

```
    content = file.read()
```

```
    print(content)
```

except FileNotFoundError:

```
    print("File does not exist")
```

32. Complete File Program

```
filename = "students.txt"
```

try:

```
with open(filename, "w") as file:
```

```
    file.write("Amit - 85\n")
```

```
    file.write("Rahul - 72\n")
```

```
    file.write("Riya - 91\n")
```

```
with open(filename, "r") as file:
```

```
    content = file.read()
```

```
    print("Student Data:")
```

```
    print(content)
```

```
except FileNotFoundError:
```

```
    print("File not found")
```

```
except PermissionError:
```

```
    print("Permission denied")
```

```
finally:
```

```
    print("File operation completed")
```

33. CSV Files

CSV means:

Comma Separated Values

Example:

id,name,marks

1,Amit,85

2,Rahul,72

3,Riya,91

Python provides the built-in:

csv

module.

34. Writing CSV

```
import csv
```

```
with open("students.csv", "w", newline="") as file:
```

```
    writer = csv.writer(file)
```

```
writer.writerow(["ID", "Name", "Marks"])
```

```
writer.writerow([1, "Amit", 85])
```

```
writer.writerow([2, "Rahul", 72])
```

```
writer.writerow([3, "Riya", 91])
```

35. Reading CSV

```
import csv
```

```
with open("students.csv", "r") as file:
```

```
    reader = csv.reader(file)
```

```
    for row in reader:
```

```
        print(row)
```

Output:

```
['ID', 'Name', 'Marks']
```

```
['1', 'Amit', '85']
```

```
['2', 'Rahul', '72']
```

```
['3', 'Riya', '91']
```

36. CSV Using Dictionary

```
import csv
```

```
with open("students.csv", "r") as file:
```

```
    reader = csv.DictReader(file)
```

```
for row in reader:
```

```
    print(row["Name"], row["Marks"])
```

Output:

Amit 85

Rahul 72

Riya 91

37. OOP — Object-Oriented Programming

OOP is a programming approach based on:

Classes

Objects

Attributes

Methods

Python supports OOP.

38. Class

A class is a **blueprint/template**.

For example:

Student

↓

Name

Age

Marks

↓

Study()

AttendClass()

GiveExam()

Python:

```
class Student:
    pass
```

39. Object

An object is an **instance of a class**.

```
class Student:
    pass
```

```
student1 = Student()
```

```
student2 = Student()
```

Here:

Student → Class

student1 → Object

student2 → Object

40. Attributes

Attributes store object data.

```
class Student:

    def __init__(self, name, age):
        self.name = name
        self.age = age
```

```
student1 = Student("Amit", 23)
```

```
print(student1.name)
```

```
print(student1.age)
```

Output:

Amit

23

41. `__init__()`

`__init__()` is called automatically when an object is created.

class Student:

```
def __init__(self, name, marks):  
    self.name = name  
    self.marks = marks
```

```
student = Student("Amit", 85)
```

```
print(student.name)
```

```
print(student.marks)
```

42. What is self?

self refers to the **current object**.

Example:

```
self.name = name
```

means:

current object's name = supplied name

For:

```
student1 = Student("Amit", 85)
```

self refers to:

student1

For:

```
student2 = Student("Rahul", 72)
```

self refers to:

student2

43. Methods

A method is a function defined inside a class.

class Student:

```
def __init__(self, name, marks):  
    self.name = name  
    self.marks = marks
```

```
def display(self):  
    print("Name:", self.name)  
    print("Marks:", self.marks)
```

```
student = Student("Amit", 85)
```

```
student.display()
```

Output:

Name: Amit

Marks: 85

44. Constructor

In beginner Python terminology, `__init__()` is commonly called the constructor.

```
class Employee:

    def __init__(self, name, salary):
        self.name = name
        self.salary = salary
```

```
emp = Employee("Amit", 30000)
```

```
print(emp.name)
```

```
print(emp.salary)
```

45. Complete OOP Example

```
class Student:

    def __init__(self, name, roll_no, marks):
        self.name = name
        self.roll_no = roll_no
        self.marks = marks

    def display(self):
        print("Name:", self.name)
        print("Roll No:", self.roll_no)
        print("Marks:", self.marks)

    def result(self):

        if self.marks >= 35:
```

```
        print("Result: PASS")
    else:
        print("Result: FAIL")
```

```
student1 = Student("Amit", 101, 85)
student2 = Student("Rahul", 102, 28)
```

```
student1.display()
student1.result()
```

```
print()
```

```
student2.display()
student2.result()
```

46. Encapsulation

Encapsulation means **keeping data and methods together inside a class and controlling access to internal data**.

Python uses naming conventions such as `_name` and `__name` for internal/private-style attributes.

Example:

```
class BankAccount:
```

```
    def __init__(self, balance):
        self.__balance = balance
```

```
    def deposit(self, amount):
        if amount > 0:
```

```
self.__balance += amount
```

```
def get_balance(self):  
    return self.__balance
```

```
account = BankAccount(5000)
```

```
account.deposit(2000)
```

```
print(account.get_balance())
```

Output:

7000

Here:

__balance

is intended to be accessed through class methods rather than directly.

47. Inheritance

Inheritance allows one class to **reuse features of another class**.

Parent Class



Child Class

Example:

class Animal:

```
def eat(self):  
    print("Animal is eating")
```

```
class Dog(Animal):
```

```
    def bark(self):
```

```
        print("Dog is barking")
```

```
dog = Dog()
```

```
dog.eat()
```

```
dog.bark()
```

Output:

Animal is eating

Dog is barking

Dog inherits:

eat()

from Animal.

48. Inheritance with `__init__()`

```
class Person:
```

```
    def __init__(self, name):
```

```
        self.name = name
```

```
    def display_name(self):
```

```
        print("Name:", self.name)
```

```
class Student(Person):

    def __init__(self, name, marks):
        super().__init__(name)
        self.marks = marks

    def display_marks(self):
        print("Marks:", self.marks)
```

```
student = Student("Amit", 85)
```

```
student.display_name()
```

```
student.display_marks()
```

Output:

Name: Amit

Marks: 85

What is super()?

super() is used to access functionality from the parent class.

```
super().__init__(name)
```

calls the parent constructor.

49. Polymorphism

Polymorphism means:

Same interface/method name can behave differently for different objects.

Example:

```
class Dog:
```

```
def sound(self):  
    print("Dog says Bark")
```

```
class Cat:
```

```
    def sound(self):  
        print("Cat says Meow")
```

```
animals = [Dog(), Cat()]
```

```
for animal in animals:
```

```
    animal.sound()
```

Output:

Dog says Bark

Cat says Meow

Both classes have:

sound()

but the behavior is different.

50. Method Overriding

Method overriding occurs when a child class provides its own implementation of a method inherited from the parent.

```
class Animal:
```

```
    def sound(self):  
        print("Animal makes sound")
```

```
class Dog(Animal):  
  
    def sound(self):  
        print("Dog barks")
```

```
dog = Dog()
```

```
dog.sound()
```

Output:

Dog barks

The child version overrides the parent version.

51. Abstraction

Abstraction means exposing **what an object does** while hiding implementation details.

Python provides abstract base classes through:

abc

Example:

```
from abc import ABC, abstractmethod
```

```
class Vehicle(ABC):
```

```
    @abstractmethod
```

```
    def start(self):
```

```
        pass
```

```
class Car(Vehicle):
```

```
    def start(self):
```

```
        print("Car starts using key/button")
```

```
class Bike(Vehicle):
```

```
    def start(self):
```

```
        print("Bike starts using self-start/kick")
```

```
car = Car()
```

```
bike = Bike()
```

```
car.start()
```

```
bike.start()
```

Output:

Car starts using key/button

Bike starts using self-start/kick

The parent class says every vehicle must have:

start()

but each child decides how start() works.

52. Four Main OOP Concepts

Concept	Simple Meaning
Encapsulation	Bundle/control data and methods
Inheritance	Reuse parent class functionality
Polymorphism	Same interface, different behavior
Abstraction	Hide implementation details

53. Mini Project — Student Management System

Now let's combine the concepts.

Project structure

```
student_management/  
|  
├── main.py  
├── student.py  
└── students.csv
```

student.py

```
class Student:
```

```
    def __init__(self, roll_no, name, marks):
```

```
        self.roll_no = roll_no
```

```
        self.name = name
```

```
        self.marks = marks
```

```
    def result(self):
```

```
        if self.marks >= 35:
```

```
            return "PASS"
```

```
return "FAIL"
```

```
def display(self):
```

```
    print("Roll No:", self.roll_no)
```

```
    print("Name:", self.name)
```

```
    print("Marks:", self.marks)
```

```
    print("Result:", self.result())
```

54. main.py

```
from student import Student
```

```
import csv
```

```
students = []
```

```
student1 = Student(101, "Amit", 85)
```

```
student2 = Student(102, "Rahul", 72)
```

```
student3 = Student(103, "Riya", 91)
```

```
students.append(student1)
```

```
students.append(student2)
```

```
students.append(student3)
```

```
for student in students:
```

```
student.display()
print("-----")
```

with open("students.csv", "w", newline="") as file:

```
writer = csv.writer(file)
```

```
writer.writerow(["Roll No", "Name", "Marks", "Result"])
```

for student in students:

```
writer.writerow([
    student.roll_no,
    student.name,
    student.marks,
    student.result()
])
```

```
print("Data saved successfully.")
```

55. Expected CSV

The generated CSV will contain:

Roll No,Name,Marks,Result

101,Amit,85,PASS

102,Rahul,72,PASS

103,Riya,91,PASS

This mini project combines:

Class

Object

__init__

self

Methods

Module

CSV

File Handling

56. Practical Project Setup

Step 1: Create folder

PythonAdvancedPractice

Step 2: Open in VS Code

Open:

File → Open Folder

Select:

PythonAdvancedPractice

Step 3: Create files

PythonAdvancedPractice/

|

├— lambda_demo.py

├— functional_demo.py

├— comprehension_demo.py

├— modules_demo.py

├— exception_demo.py

├— file_demo.py

├— csv_demo.py

└─ oop_demo.py

└─ student_management/

57. Running Python Programs

Open VS Code terminal.

Check Python:

```
python --version
```

Run a program:

```
python lambda_demo.py
```

For the project:

```
cd student_management
```

```
python main.py
```

58. Beginner Practice Assignments

Assignment 1 — Lambda

Create lambda functions for:

1. Square of a number
2. Cube of a number
3. Add two numbers
4. Check even/odd

Solution

```
square = lambda x: x * x
```

```
cube = lambda x: x * x * x
```

```
add = lambda a, b: a + b
```

```
even_odd = lambda x: "Even" if x % 2 == 0 else "Odd"
```

```
print(square(5))
```

```
print(cube(3))
```

```
print(add(10, 20))  
print(even_odd(8))
```

59. Assignment 2 — map()

Given:

```
numbers = [2, 4, 6, 8, 10]
```

Create a new list containing their squares.

Solution

```
numbers = [2, 4, 6, 8, 10]
```

```
result = list(map(lambda x: x * x, numbers))
```

```
print(result)
```

Output:

```
[4, 16, 36, 64, 100]
```

60. Assignment 3 — filter()

Given:

```
numbers = [10, 15, 20, 25, 30, 35, 40]
```

Find numbers divisible by 5.

Solution

```
numbers = [10, 15, 20, 25, 30, 35, 40]
```

```
result = list(filter(lambda x: x % 5 == 0, numbers))
```

```
print(result)
```

61. Assignment 4 — reduce()

Find the sum of:

[10, 20, 30, 40, 50]

Solution

```
from functools import reduce
```

```
numbers = [10, 20, 30, 40, 50]
```

```
result = reduce(lambda a, b: a + b, numbers)
```

```
print(result)
```

Output:

150

62. Assignment 5 — List Comprehension

Create a list containing cubes from 1 to 10.

Solution

```
cubes = [x ** 3 for x in range(1, 11)]
```

```
print(cubes)
```

63. Assignment 6 — Dictionary Comprehension

Create:

1 → 1

2 → 4

3 → 9

...

10 → 100

Solution

```
squares = {  
    x: x * x  
    for x in range(1, 11)  
}  
  
print(squares)
```

64. Assignment 7 — Exception Handling

Ask the user for two numbers and divide them.

Handle:

- Invalid input
- Division by zero

Solution

try:

```
a = int(input("Enter first number: "))  
b = int(input("Enter second number: "))
```

```
result = a / b
```

except ValueError:

```
    print("Please enter numbers only")
```

except ZeroDivisionError:

```
    print("Cannot divide by zero")
```

else:

```
    print("Result:", result)
```

finally:

```
print("Program completed")
```

65. Assignment 8 — File Handling

Create a program that:

1. Creates employees.txt
2. Writes 5 employee names
3. Reads the file
4. Displays the names

Solution

with open("employees.txt", "w") as file:

```
file.write("Amit\n")
```

```
file.write("Rahul\n")
```

```
file.write("Riya\n")
```

```
file.write("Ankit\n")
```

```
file.write("Priya\n")
```

with open("employees.txt", "r") as file:

```
content = file.read()
```

```
print(content)
```

66. Assignment 9 — OOP

Create a Book class containing:

title

author

price

Create a method:

display()

Solution

class Book:

```
def __init__(self, title, author, price):
```

```
    self.title = title
```

```
    self.author = author
```

```
    self.price = price
```

```
def display(self):
```

```
    print("Title:", self.title)
```

```
    print("Author:", self.author)
```

```
    print("Price:", self.price)
```

```
book1 = Book(
```

```
    "Python Programming",
```

```
    "ABC",
```

```
    500
```

```
)
```

```
book1.display()
```

Create:

Person



Student

Person should contain:

name

Student should contain:

marks

Solution

```
class Person:
```

```
    def __init__(self, name):
```

```
        self.name = name
```

```
    def display_name(self):
```

```
        print("Name:", self.name)
```

```
class Student(Person):
```

```
    def __init__(self, name, marks):
```

```
        super().__init__(name)
```

```
        self.marks = marks
```

```
    def display(self):
```

```
        print("Name:", self.name)
```

```
        print("Marks:", self.marks)
```

```
student = Student("Amit", 85)
```

```
student.display()
```

68. Assignment 11 — Polymorphism

Create classes:

Dog

Cat

Cow

Each class should have:

sound()

Solution

```
class Dog:
```

```
    def sound(self):
```

```
        print("Dog: Bark")
```

```
class Cat:
```

```
    def sound(self):
```

```
        print("Cat: Meow")
```

```
class Cow:
```

```
    def sound(self):
```

```
print("Cow: Moo")
```

```
animals = [Dog(), Cat(), Cow()]
```

```
for animal in animals:
```

```
    animal.sound()
```

69. Assignment 12 — Combined Intern-Level Task

Create an **Employee Management System** that:

- Uses a class
- Stores employee objects
- Uses inheritance
- Uses a method
- Saves employees to CSV
- Reads CSV
- Handles file exceptions
- Uses list comprehension to find employees with salary > ₹30,000

Expected structure

employee_management/

|

├— main.py

├— employee.py

└— employees.csv

This is a good **intern-level practice project** because it combines multiple Python concepts instead of testing each concept independently.

70. Quick Revision Sheet

Lambda

```
lambda x: x * 2
```

map

```
list(map(lambda x: x * 2, numbers))
```

filter

```
list(filter(lambda x: x > 10, numbers))
```

reduce

```
from functools import reduce
```

```
reduce(lambda a, b: a + b, numbers)
```

List comprehension

```
[x * 2 for x in numbers]
```

Dictionary comprehension

```
{x: x * x for x in numbers}
```

Module

```
import math
```

Package

```
from utilities.calculator import add
```

Exception

```
try:
```

```
    ...
```

```
except ValueError:
```

```
    ...
```

File

```
with open("data.txt", "r") as file:
```

```
    data = file.read()
```

Class

```
class Student:
```

...

Constructor

```
def __init__(self):
```

...

Object

```
student = Student()
```

Inheritance

```
class Student(Person):
```

...

Parent constructor

```
super().__init__()
```

Method overriding

```
class Child(Parent):
```

```
    def show(self):
```

...

Abstraction

```
from abc import ABC, abstractmethod
```

71. Final Learning Challenge

After completing all the above assignments, build this project:

Student Result Management System

Features:

1. Add Student
2. View Students
3. Search Student
4. Calculate Result
5. Find Passed Students

6. Find Failed Students
7. Find Topper
8. Calculate Average Marks
9. Save to CSV
10. Load from CSV
11. Handle invalid input
12. Handle missing file

Try to use:

- ✓ Functions
- ✓ Lambda
- ✓ map()
- ✓ filter()
- ✓ reduce()
- ✓ List comprehension
- ✓ Dictionary comprehension
- ✓ Modules
- ✓ Packages
- ✓ try/except
- ✓ raise
- ✓ with open()
- ✓ CSV
- ✓ Classes
- ✓ Objects
- ✓ __init__()
- ✓ self
- ✓ Encapsulation
- ✓ Inheritance

✓ Polymorphism

✓ Abstraction

✓ Method overriding

This progression takes you from **basic Python functional programming → file/data handling → OOP → a practical intern-level project**, which is a strong foundation before moving into frameworks such as **Django, Flask, FastAPI, NumPy/Pandas, or backend development**.